

Final Project

Particle Filter Localization with AR Tags in Simulation

Due: second week of finals

1 Overview

In this project the student will implement a **particle filter** to estimate the pose of a mobile robot inside a simulated room. The room contains AR tags placed on its walls, and the robot uses its on-board camera to detect them. Because the tags all share the same ID, the robot does not know which tag it is observing, and the filter must handle this ambiguity. The robot is moved manually (teleop) inside the simulation; localization runs in real time and the result is visualized as a particle cloud.

2 Simulation Environment

- Use **Gazebo** as the simulator.
- Build a closed rectangular room. The room dimensions are chosen by the student.
- Place **8 AR tags** on the walls (2 on each wall). All tags must share the same ID.
- The tags must be placed **asymmetrically**, so that the configuration of tag positions is unique to each part of the room. This asymmetry is what allows the filter to converge over time.
- All AR tags must have the **same physical size**; the size itself is chosen by the student.
- The position of every tag in the world frame must be known to the filter (the filter has access to the tag map). What is not known is which physical tag the camera is currently seeing.
- Add a mobile robot with a forward-facing camera. The robot is driven manually using teleop; the filter consumes its odometry as the motion input.

3 Tasks

3.1 Task 1: Particle Filter Implementation

- Place the robot at an unknown location in the room.
- Drive the robot manually and run the particle filter to estimate its pose.
- Each particle represents a hypothesis of the robot pose (x, y, θ) .

3.2 Task 2: Multi-Hypothesis Sensor Model

Because all AR tags share the same ID, an observation does not tell the filter which tag was seen. The sensor model must therefore consider **all 8 tags as possible sources** of the observation.

- For each particle, compute the likelihood of the observation by summing over all tag hypotheses:

$$p(z \mid x_{\text{particle}}) = \sum_{i=1}^8 p(z \mid x_{\text{particle}}, \text{tag}_i).$$

- Do **not** simplify this by picking only the nearest tag to the particle. That shortcut breaks the filter and is not accepted.
- The expected behavior is that, early in the run, several plausible particle clusters may survive (because the room layout is partially symmetric from a single observation), and as the robot moves and observes more tags, the wrong clusters are eliminated and the cloud collapses around the true pose.

3.3 Task 3: Visualization

The student must visualize the run in real time. The visualization must show, in the same view:

- The room and the AR tag positions.
- The **particle cloud** (all particles, colored by weight).
- The robot's **odometry-only trajectory** (what the robot would think its path is if it trusted odometry alone).
- The **particle filter pose estimate** trajectory (e.g. the weighted mean of the particles) over time.

Showing the odometry and the filter estimate side by side makes the benefit of the filter visible.

4 Report

The report must include, **at minimum**:

- A description of the simulation setup (room, tag layout, robot, camera).
- The motion model and the noise parameters used in the prediction step.
- The sensor model, including how the multi-hypothesis likelihood is computed.
- A short discussion of how the particle filter relates to **Bayesian filtering** (prior, likelihood, posterior, and how each step of the filter corresponds to these quantities).
- Screenshots or short videos of the particle cloud at different stages: initial spread, partial convergence, and final converged state.
- A GitHub link to the full source code of the project.

5 Presentation

Each group will give a short presentation to **Dr. Özgür Erkent** and research assistant **Mehmet Muratoğlu**. In your presentation, explain what you did for the project and include videos of your simulation experiments.

6 Submission

- Submit a single **.zip** archive containing the report and the presentation slides.
- The full source code is not placed in the **.zip**; instead, the report must include a GitHub link to the project repository.

7 Bonus — Real Duckiebot

Students who want extra credit may also run the same particle filter on a real **Duckiebot**. The simulation is still required; the Duckiebot run is on top of it. The same multi-hypothesis sensor model and the same visualization requirements apply. A short demo video of the real run must be included in the report.

8 Grading

The project is graded on three components:

- **Code** — correctness and quality of the particle filter implementation.
- **Report** — clarity, completeness, and the discussion of the Bayesian interpretation.
- **Presentation** — clarity of the explanation and ability to answer questions.